

Conference Paper Title*

*Note: Sub-titles are not captured in Xplore and should not be used

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

4th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

5th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

6th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

Abstract—
Index Terms—

I. INTRODUCTION

Agent skills are increasingly used to package reusable capabilities for LLM-based coding agents. In practice, a skill is rarely just a single prompt file. It often combines instruction text, helper scripts, configuration, examples, and external resources. This makes it look like a small software artifact, but one whose interface is partly natural language and whose operational boundary is entangled with agent runtime behavior. The central premise of this paper is that skills should be studied from both a software composition perspective and a security evaluation perspective. Without the first, security discussions stay too abstract. Without the second, structural observations remain detached from practical attack and defense questions.

The paper is organized around three research questions. The first asks how skills are composed in the wild and what their essential components are. This question is meant to establish a stable descriptive baseline. The second asks what attack vectors and attack surfaces are relevant to skills. This question is intended to refine the current understanding of skill security by connecting prior work on prompt injection, supply chain abuse, and agent tooling to the concrete structure of the skill artifact. The third asks what the current battlefield between attacks and defenses looks like. This question shifts from description and taxonomy to comparative evaluation, using representative attacks and existing scanners to understand which parts of the skill artifact are well covered and which are still weakly defended.

The full paper will therefore proceed in three layers. It will first characterize the artifact. It will then organize the threat space around that artifact. It will finally evaluate how current defensive tools behave when attacks are instantiated against

different skill components. At the current stage, the purpose of this draft is to fix a clear and writable outline for those three layers, so that later iterations can fill in measurements, tables, and discussion without having to repeatedly change the overall argument.

II. BACKGROUND AND SCOPE

This section will frame skills as a distinct software artifact. The first part will define what counts as a skill in practice, including the role of instruction files such as `SKILL.md`, auxiliary scripts, metadata, manifests, embedded examples, and references to remote resources. The goal is to show that a skill is neither purely a prompt nor purely a package. It is a hybrid artifact whose composition matters for both usability and security.

The second part of this section will clarify the paper's scope. The focus will be on attacks that are carried by the skill artifact itself or that are directly enabled by how the artifact is loaded and used. This includes malicious instructions, deceptive documentation, script-level payloads, misleading declarations, and attacks that exploit the relationship between artifact content and agent action. By contrast, the paper will not attempt to cover model pretraining attacks, general web prompt injection unrelated to skills, or broad runtime security problems that do not depend on the skill channel. This scoping is important because the space around agent security is already large, and the paper needs a clear center of gravity.

III. METHODOLOGY

The methodology section will explain how the three research questions fit into one coherent study design. The study begins with corpus construction. We plan to collect roughly one thousand skills from multiple sources, including public marketplaces and repository-based distributions, while preserving variation in popularity and publisher type. The objective is not to build a complete census of the ecosystem but

Identify applicable funding agency here. If none, delete this.

to construct a corpus that is broad enough to reveal recurring structural patterns.

After corpus construction, the paper will perform a structural analysis of the sampled skills. This part supports RQ1. We will inspect file types, directory layouts, presence of scripts, presence of configuration artifacts, and references to external dependencies or services. We will then compare these observations with classic software architecture concepts such as interface, implementation, dependency, and packaging. This step is important because the paper is not only asking what skills contain, but also how that composition should be interpreted.

The next methodological component is a lightweight literature review, which supports RQ2. The review will use recent work on skill security, agent security, prompt injection, and transferable supply-chain attacks as the main search area, followed by one round of backward snowballing for highly relevant papers and tools. The purpose is not to claim a complete systematic review. The purpose is to build a defensible and hierarchical account of attack vectors that is sufficiently grounded in prior work while remaining directly tied to the skill artifact observed in RQ1.

The final methodological component is the evaluation protocol for RQ3. The current benchmark and experimental results will be treated as the starting point. We will organize existing attack cases by the component they target, and where necessary we will add manipulated variants that move the same malicious intent across instructions, scripts, metadata, and external references. Existing scanners and model-backed analyzers will then be run under a unified harness. The main output will be a comparative view of what present defenses can and cannot detect under the current setup, together with a discussion of how much of that behavior can be explained by skill composition itself.

IV. RQ1: HOW ARE SKILLS COMPOSED AS A SOFTWARE ARTIFACT

The first research question is intended to provide the descriptive foundation of the paper. This section will begin by quantifying which major components appear in the sampled corpus. It will examine how often skills contain only documentation, how often they include executable files, how often they ship with supporting examples or references, and how often they declare or imply interaction with external tools and services. Rather than presenting these observations as an unstructured inventory, the section will use them to identify a small number of recurring structural archetypes.

The second part of the section will interpret these archetypes from a software artifact perspective. A documentation-heavy skill may function more like a natural-language interface wrapper. A skill with multiple scripts and auxiliary files may resemble a lightweight plugin package. A skill with strong dependence on external endpoints may behave more like a configuration stub that delegates critical behavior elsewhere. The point of this analysis is to explain what remains familiar

from traditional software packaging and what is genuinely new in the skill setting.

The final part of RQ1 will set up the later sections by arguing that the composition of the artifact is not merely descriptive background. It is also the basis for how attacks can be hidden, how permissions can be misrepresented, and how scanners may fail. In that sense, RQ1 is the structural layer that the rest of the paper will depend on.

V. RQ2: WHAT ARE THE ATTACK VECTORS AND ATTACK SURFACES FOR SKILLS

The second research question will convert the current flat account of skill attacks into a more explicit hierarchical structure. This section will first revisit the most relevant prior work and separate three sources of attack knowledge: attacks designed specifically for skills, attacks originally framed for agents or prompt injection but directly transferable to skills, and attacks adapted from traditional software supply chains. This separation is necessary because the paper should not present all attack vectors as equally native to the skill setting.

The next part of the section will organize the attack space by linking attack goals to artifact components and triggering conditions. At a high level, the taxonomy will distinguish where the attack is embedded, what capability it tries to obtain, and under what conditions it becomes effective. This hierarchical structure is meant to improve over a simple list of attack names. It should show not only that multiple attack families exist, but also how they relate to the composition patterns identified in RQ1.

The final part of RQ2 will explain how this taxonomy will be used later in the paper. It will serve as the conceptual bridge between corpus characterization and empirical evaluation. It will define what kinds of attack placement are meaningful to test, what kinds of scanner coverage claims can reasonably be made, and where the current literature still appears incomplete. This makes RQ2 more than a survey section. It is the organizing theory for the later evaluation.

VI. RQ3: WHAT DOES THE CURRENT ATTACK AND DEFENSE BATTLEFIELD LOOK LIKE

The third research question is the paper’s empirical confrontation section. It will take representative attacks and position them against current scanners and model-backed analyzers. The purpose is not simply to produce a leaderboard. The purpose is to observe how defense behavior changes when attacks are placed in different parts of the skill artifact, and whether some skill components are consistently easier or harder to defend than others.

This section will begin by describing the attack set that is already available and how it will be organized for evaluation. Existing attack cases provide a practical starting point, but the section will also make room for manipulated variants that preserve the same malicious intent while changing the carrier component. For example, an attack originally expressed in instruction text may be recast through auxiliary scripts or through misleading metadata. This design will allow the

paper to connect empirical outcomes back to the artifact model developed in RQ1 and the attack hierarchy developed in RQ2.

The next part will describe the evaluation targets. The plan is to test multiple scanners and model-backed settings, including less capable baselines where feasible, under a common protocol. The key question is not only which system detects more cases overall, but also which kinds of attacks remain detectable once the same intent is moved across different components. This is where the paper can make a useful claim even if absolute scanner performance turns out to be stronger than initially expected.

The final part of RQ3 will frame the results as a battlefield rather than a static benchmark report. It will discuss where existing defenses appear robust, where they appear component-sensitive, and where the current benchmark remains too weak to support strong claims. This section should therefore end with a balanced interpretation: either the evidence supports a meaningful security evaluation paper, or it reveals that the contribution is narrower and should be framed more modestly.

VII. DISCUSSION

VIII. CONCLUSION

ACKNOWLEDGMENT

REFERENCES

REFERENCES

- [1] References will be inserted after the empirical sections and literature review are finalized.